

JTressette

Dati Studente

Matricola: 2082332

Nome: Matteo

Cognome: Mei

Corso: Informatica (M-Z)

Introduzione

Il progetto JTressette è stato sviluppato utilizzando l'ambiente di sviluppo Eclipse e interamente scritto in linguaggio Java, con l'obiettivo di ricreare il gioco di carte italiano "Tressette" in un ambiente interattivo e immersivo. Il design dell'applicazione segue il modello architetturale Model-View-Controller (MVC), affiancato dal pattern Observer/Observable, per garantire una comunicazione fluida ed efficiente tra i componenti.

Dal punto di vista grafico, l'interfaccia è stata progettata principalmente utilizzando le funzionalità native di Swing, con particolare attenzione alla personalizzazione della visualizzazione delle carte tramite immagini. Per migliorare l'esperienza dell'utente sono stati integrati elementi sonori, con file audio posizionati nella directory dedicata /res/audio

Specifiche

Menu principale

La schermata del menu principale funge da punto di accesso centrale per tutte le funzionalità del gioco, ed è progettata per garantire un'esperienza semplice ed intuitiva. Qui l'utente può navigare rapidamente tra le opzioni "Gioca", "Regole", "Profilo", "Impostazioni" ed "Esci", selezionando visivamente l'opzione desiderata tramite un cursore personalizzato che evidenzia la voce attiva.

In essa ho implementato il pattern MVC, garantendo una separazione chiara tra visualizzazione, logica e dati. Il Model è rappresentato dalla classe `Cursor`, che gestisce il cursore di navigazione e traccia l'indice della voce del menu attualmente selezionata. La View, attraverso `MenuFrame`, visualizza le voci del menu con uno stile chiaro, adottando un design grafico moderno che include gradienti, trasparenze e pannelli con uno stile che verrà poi ripetuto e riutilizzato in tutta l'applicazione. Il Controller, gestito da JTressette, si occupa di interpretare l'input dell'utente (click del mouse) e di propagare le selezioni alle opzioni del menu.



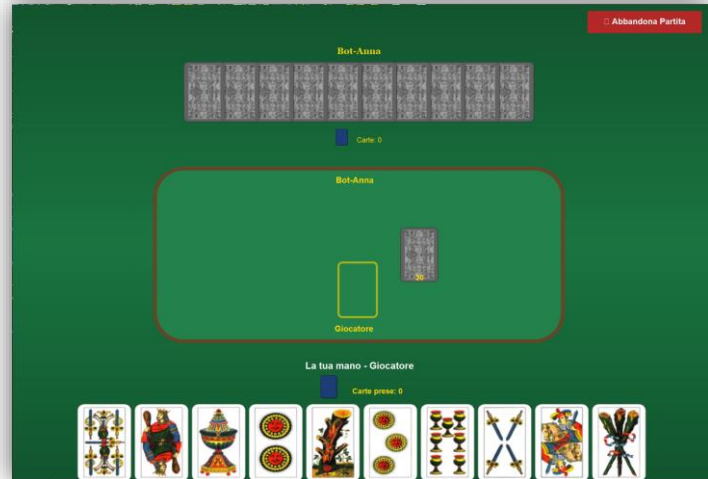
Partita

La gestione delle partite ha puntato alla creazione di un sistema fluido e immersivo, in grado di consentire a un giocatore umano di sfidare da uno a tre giocatori artificiali (*bot*), con livelli di difficoltà personalizzabili (EASY, MEDIUM, HARD): tutto ciò utilizzando, anche in questa sezione, il modello MVC ed il pattern Observer/Observable.

La partita inizia con un'interfaccia di configurazione che permette all'utente di scegliere il numero di avversari, assegnare loro nomi personalizzati o generarli automaticamente, e selezionare il livello di difficoltà.

Il *Model*, rappresentato dalla classe GameState, mantiene lo stato della partita, gestendo mazzi, mani dei giocatori, punteggi e carte giocate. Si occupa inoltre di applicare le regole del Tressette, come la determinazione delle prese e dei vincitori. I giocatori sono modellati attraverso le classi Giocatore, GiocatoreUmano e Bot, che differenziano i comportamenti tra giocatore umano e giocatore artificiale, con quest'ultimo che utilizza strategie dinamiche basate sul livello di difficoltà. Le classi Mazzo e Cards integrano le regole delle carte con i ranghi e i semi definiti.

La View, realizzata nella classe GameView, si occupa di rappresentare graficamente il tavolo di gioco e le mani dei giocatori, mostrando inoltre animazioni per la distribuzione delle carte, il loro movimento durante i turni e il feedback visivo per le prese vinte. Elementi come log delle azioni e indicatori di punteggio migliorano l'interattività e l'esperienza utente. Grazie al meccanismo Observer, la classe GameView si aggiorna dinamicamente in risposta ai cambiamenti nello stato della partita, garantendo coerenza tra logica e interfaccia.



Il Controller, implementato con la classe GameController, gestisce il flusso principale della partita, sincronizzando le interazioni tra GameState e GameView. Tale classe coordina il ciclo di gioco, che include la distribuzione delle carte, i turni dei giocatori, le mosse strategiche dei bot e il calcolo delle prese vinte. Inoltre, grazie al pattern Observer, si interfaccia con le impostazioni del gioco (via MenuImpostazioni), regolando ad esempio gli effetti sonori o la visibilità dei punteggi durante la partita.

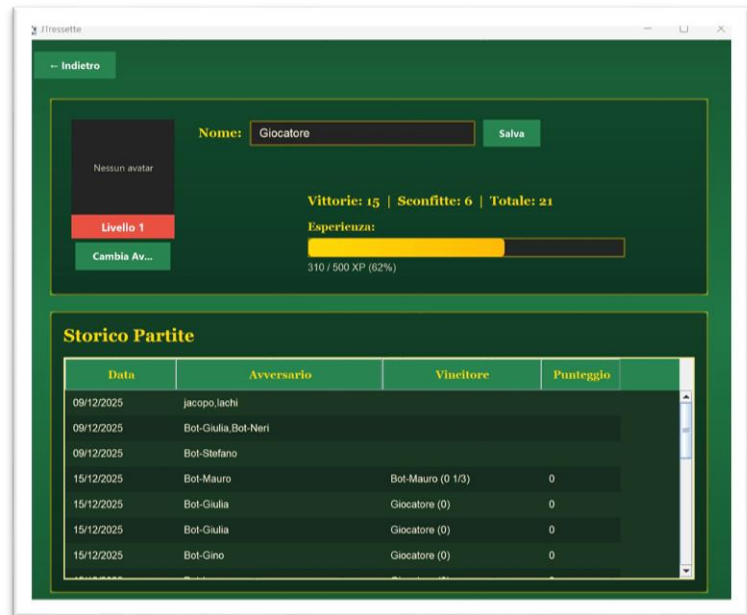
Al termine del gioco, il sistema registra tutte le informazioni della partita nel profilo dell'utente, inclusi avversari, punteggi e risultati, sincronizzando automaticamente i dati grazie al meccanismo Observable.

Gestione del profilo

L'obiettivo per la gestione del profilo è stato quello di ottimizzare l'esperienza utente, dando rilievo alla visualizzazione del nome e dell'immagine del proprio profilo già nella schermata principale (Home), posizionati in alto a destra. Questi sono resi interattivi per un accesso rapido alla schermata dettagliata del profilo tramite un clic o l'opzione "Profilo" nel menu principale.

La schermata del profilo permette all'utente di modificare il proprio nome, scegliere un nuovo avatar, tramite la selezione di un file dal proprio pc, e consultare statistiche dettagliate come il numero di partite giocate, vittorie, livello e punti esperienza accumulati. È inoltre possibile visualizzare lo storico delle partite con informazioni sugli avversari, risultati e punteggi, garantendo all'utente una panoramica completa del proprio progresso nel gioco.

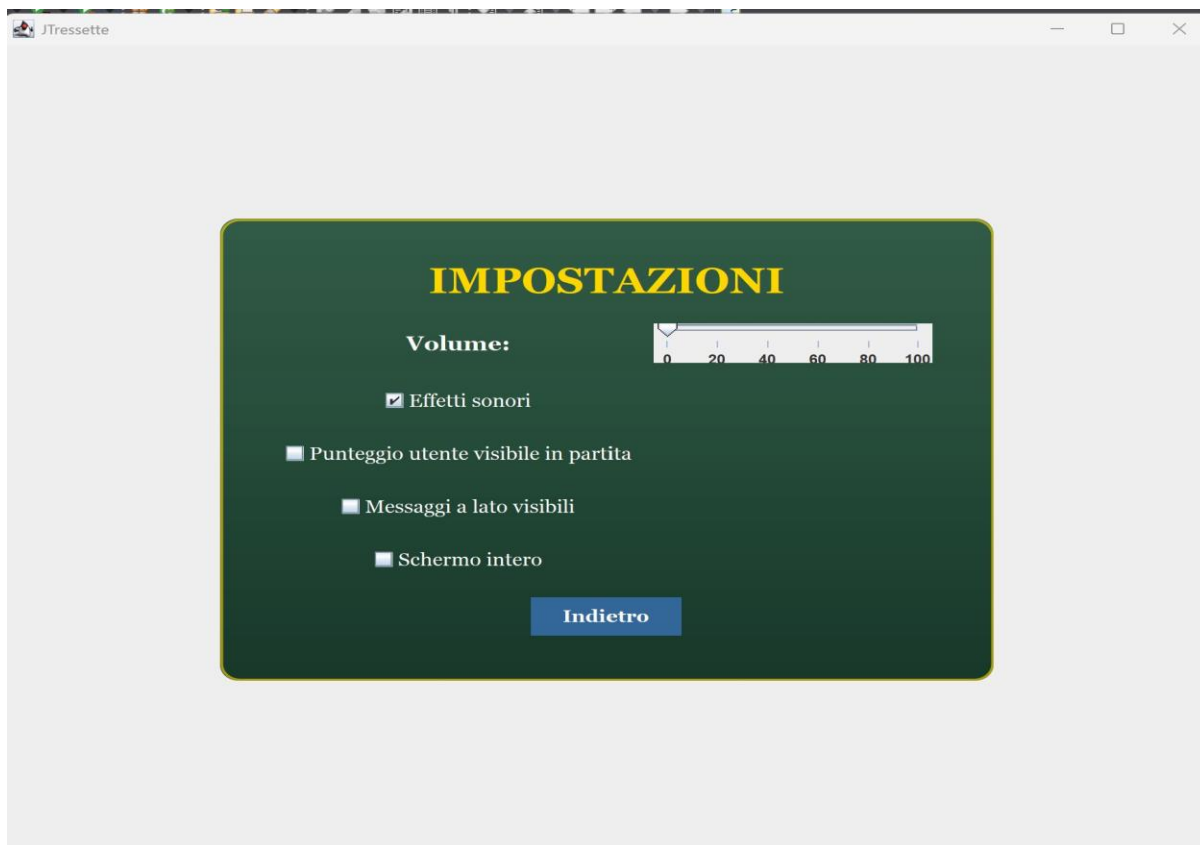
Ogni utente che vi accede al gioco avrà il proprio livello di esperienza, determinato dal numero di carte prese, dal numero di punti fatto e dall'esito delle partite da lui giocate. Inoltre, è visibile il suo progresso attraverso una barra visiva presente sempre nella pagina del profilo. I dati sono gestiti in modo persistente attraverso un file salvato localmente e sono aggiornati dinamicamente grazie al pattern Observer, consentendo una sincronizzazione in tempo reale tra le modifiche dei dati e la loro rappresentazione grafica. Inoltre, l'interfaccia adotta uno stile moderno, mantenendo colori e layout coerenti con la schermata principale per un'ottima usabilità.



Impostazioni

La pagina delle impostazioni consente all'utente di personalizzare vari aspetti dell'esperienza di gioco, come il volume dell'audio, quale musica di background o effetti di gioco (tramite slider), l'abilitazione dei quest'ultimi, la visualizzazione dei punteggi e dei messaggi durante la partita, oltre ad avere un'opzione per la modalità full screen. Grazie al pattern Singleton, la classe MenuImpostazioni centralizza la gestione di tutti i parametri, garantendo consistenza tra sessioni di gioco.

Per migliorare la reattività, le modifiche effettuate vengono propagate immediatamente a tutti i componenti interessati tramite il pattern Observer, come l'audio o la grafica. La schermata delle impostazioni è stata progettata con uno stile moderno, mantenendo una coerenza visiva con il resto dell'applicazione grazie all'utilizzo di gradienti, trasparenze e pannelli verdi.



Regole del gioco

La pagina delle regole fornisce una guida chiara e dettagliata sulle regole del Tressette, accessibile facilmente dal menu principale. Questa schermata è studiata per essere semplice ed esplicativa, supportando sia i nuovi giocatori che quelli esperti.

Audio

L'audio è stato integrato per arricchire l'esperienza utente, utilizzando la classe *AudioManager*, che centralizza la riproduzione e la gestione dei file audio. Grazie al pattern *Observer*, ad esempio, il controllo del volume e la disabilitazione degli effetti sonori sono sincronizzati dinamicamente con l'interfaccia grafica. Le animazioni sono realizzate tramite *Swing* e *Timer*, per garantire interattività e fluidità nei movimenti delle carte e nei feedback visivi. La sincronizzazione delle animazioni con la logica di gioco è mantenuta usando *SwingUtilities.invokeLater* per operazioni grafiche su thread dedicati.



Ulteriori implementazioni

Oltre al modello architetturale *MVC* e al pattern *Observer/Observable*, il progetto JTressette sfrutta altre tecnologie e pattern per ottimizzare il design e la funzionalità del codice. Due aspetti fondamentali sono l'uso di *Java Swing*, per la creazione dell'interfaccia grafica, e l'applicazione delle *Stream API* di Java per la manipolazione dei dati.

Java Swing

Utilizzato come framework centrale per lo sviluppo dell'interfaccia grafica. Tutte le schermate visive del progetto, come il menu principale, la configurazione delle partite, la rappresentazione del tavolo da gioco e la gestione del profilo, sono state implementate utilizzando Swing. Le classi principali che compongono l'interfaccia, come *GameView*, *ProfileMenu*, *ViewImpostazioni* e *MenuFrame*, fanno uso di componenti Swing per creare una GUI interattiva e moderna.

Grazie a Java Swing, è stato possibile utilizzare elementi visivi come *JPanel*, *JSlider*, *JCheckBox*, *JTable* e *JLabel*, combinati con layout personalizzati come *BorderLayout*, *FlowLayout* e *CardLayout*. Questi componenti sono stati estesi per includere animazioni e gradienti visivi eleganti, garantendo una coerenza grafica tra le diverse sezioni. Ad esempio, nel menu principale e nelle impostazioni, pannelli traslucidi e gradienti verdi sono stati implementati sovrascrivendo il metodo *paintComponent*.

Stream API

Le Stream API di Java sono utilizzate per manipolare collettivamente dati come carte, giocatori e punteggi in modo efficiente. Queste API consentono di evitare costrutti iterativi complessi, migliorando la leggibilità e l'efficienza del codice.

Un esempio rilevante è la classe *Bot*, dove gli Stream vengono utilizzati per calcolare mosse strategiche in base al livello di difficoltà. Per il livello *MEDIUM*, ad esempio, uno Stream filtra le carte legali per selezionare quella che vince la presa corrente o quella meno importante da scartare. Anche nella classe *GameState*, gli Stream sono essenziali per operazioni come il calcolo dei punti delle prese o la determinazione del vincitore di un turno.

Nella generazione del mazzo, la classe *Mazzo* utilizza Stream per creare le 40 carte necessarie, combinando semi e ranghi in modo efficiente. Inoltre, gli Stream sono utilizzati per analisi logiche (es. verifica di condizioni su collezioni) e per filtrare dati dinamici, come identificare le carte ancora utili nel gioco o ottimizzare la distribuzione delle carte.

Conclusione

JTressette combina una progettazione rigorosa e funzionalità avanzate per offrire un'esperienza completa e coinvolgente, rispettando le specifiche richieste e aggiungendo elementi personali di design. Ogni componente, dall'estetica all'interazione dinamica, riflette un approccio professionale e modulare.